Deep Agents: Concepts, Architecture, and Application to the LUMS Process Orchestrator

A study reference on LangChain's Deep Agents pattern — intuition, LangGraph foundations, context and memory engineering, real-world use, and how it maps onto our own system design

Compiled September 29, 2026 · Based on LangChain Academy and official LangChain/LangGraph documentation

Contents

1. Intuition — what makes an agent “deep”
2. The stack: LangChain, LangGraph, and Deep Agents
3. Where the pattern came from
4. The four pillars of a deep agent
5. Context engineering: write, select, compress, isolate
6. Memory: short-term vs. long-term
7. How a deep agent actually runs — a walkthrough
8. The LangChain Academy Deep Agents course — structure and study order
9. A real-world example: Open Deep Research (and Claude Code)
10. Technical tie-in: mapping deep agents onto the LUMS Process Orchestrator
11. Deployment: how this would actually run in production
12. Honest trade-offs — is the full framework worth adopting for our MVP?
13. Glossary
14. Sources

1. Intuition — What Makes an Agent “Deep”

Most people's first experience of an "AI agent" is a single-turn tool-calling loop: the model gets a question, decides to call a tool (search the web, run a calculation), reads the result, and answers. That works well for short tasks, but it breaks down the moment a task takes many steps, spans a long time, or requires the agent to remember what it already tried three steps ago.

A **deep agent** is the name LangChain gives to an agent built specifically to survive that kind of task: one that *plans* before acting, *persists* its state so it can resume after a pause (sometimes days later), *delegates* heavy sub-tasks so they don't clutter its main train of thought, and knows when to *stop and ask a human* before doing something consequential. The name is deliberately contrasted with a "shallow" agent — one that can call tools but has no memory of a plan, no way to hand off work, and no persistent state across a pause.

This distinction is not academic for our project. Force enrollment is not a single-turn question — it is a request that has to survive a multi-day wait for an instructor's approval, then resume exactly where it left off. That is precisely the shape of task deep agents are built for, which is why this is worth understanding properly rather than skimming.

2. The Stack: LangChain, LangGraph, and Deep Agents

These three names are often used interchangeably in casual conversation, but they are three distinct layers, each built on the one below it:

Layer	What it actually is
LangChain	The foundational toolkit: standardized ways to call any LLM, define tools, format messages, and connect to retrieval sources. On its own, it does not manage long-running state — it is the building-block library.
LangGraph	A durable execution runtime built on top of LangChain. It represents an agent as a graph of steps, and its core value is **persistence**: it can checkpoint the exact state of a run, pause it (for a human-in-the-loop approval, for example), and resume it later — even after the process restarts. This is the piece that makes long-running, interruptible workflows possible at all.
Deep Agents	A pre-built "agent harness" on top of LangGraph. It does not invent new capability — it packages LangGraph's persistence together with a planning tool, a virtual filesystem, sub-agent delegation, and human-in-the-loop interrupts into one ready-made scaffold, so you don't have to hand-wire all of that yourself for every project.

> A useful mental model: LangChain gives you the parts, LangGraph gives you the engine that keeps running reliably even when paused, and Deep Agents gives you a pre-assembled car built from both, tuned specifically for long, complex tasks.

3. Where the Pattern Came From

The Deep Agents pattern was not invented in the abstract. LangChain's own engineering team built it after studying three AI applications that were already succeeding at long, complex, autonomous tasks: **Deep Research** (OpenAI's and others' long-form research agents), **Manus** (a general-purpose autonomous task agent), and **Claude Code** — the same class of coding agent underlying the tool used to research and write this document.

The team's specific observation about Claude Code is worth sitting with: people were using it for far more than coding — project planning, research, file organization — and the question became *what architectural choices made it that versatile?* The answer they landed on became the four pillars in the next section.

4. The Four Pillars of a Deep Agent

4.1 A Detailed System Prompt

Deep agents lean on long, carefully written system prompts — explicit instructions on how to use each tool, plus worked examples ("few-shot" demonstrations) of how to behave in specific situations. This is unglamorous but load-bearing: a lot of what looks like "reasoning ability" in a strong agent is actually well-specified behavior baked into the prompt.

4.2 A Planning Tool

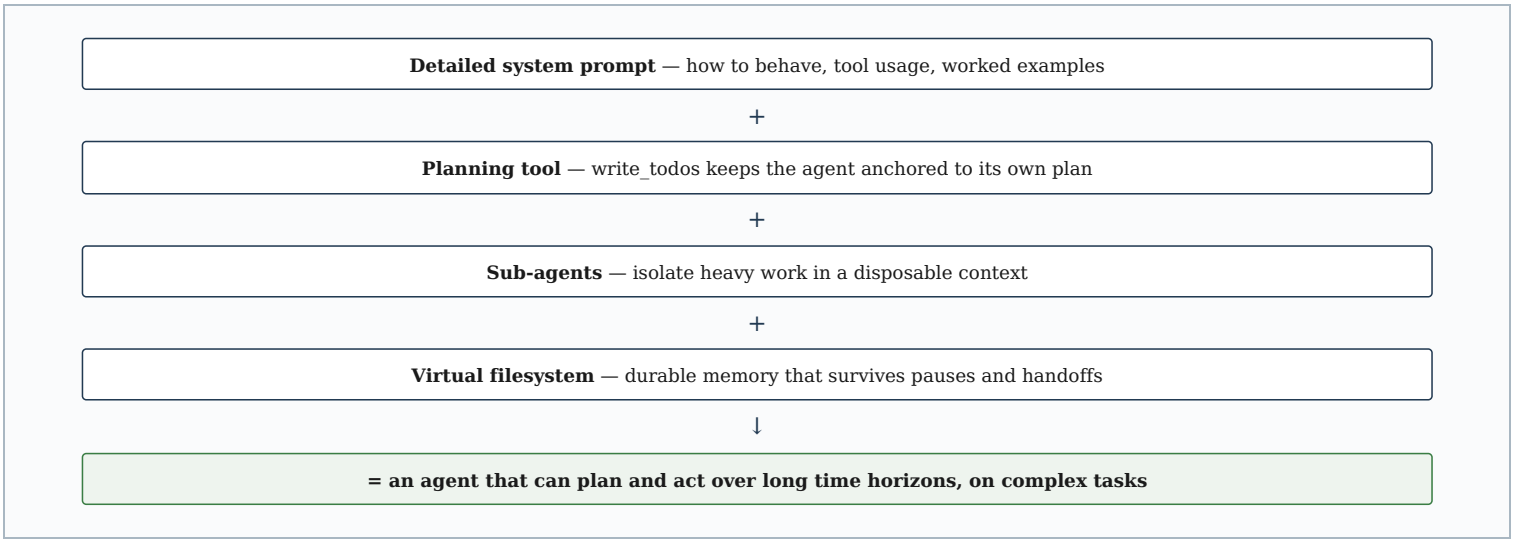
Deep agents carry an explicit `write_todos`-style tool that lets the agent lay out its steps before acting, and check them off as it goes. The genuinely surprising finding from LangChain's own analysis of Claude Code: this tool is a **no-op** — it doesn't execute anything or change any state. Its entire value is that writing the plan down keeps the agent's own attention anchored to the plan as the conversation grows, instead of drifting. It is a context-engineering trick disguised as a to-do list.

4.3 Sub-Agents

A deep agent can spawn **sub-agents**: short-lived child agents with their own clean context window, given one narrow task, that report back a single, condensed result. This solves a real failure mode — if the main agent has to personally read 40 pages of search results to answer one sub-question, all 40 pages sit in its context for the rest of the task, crowding out everything else it needs to remember. A sub-agent absorbs that mess and only the distilled answer comes back.

4.4 A Shared File System

Deep agents get a **virtual filesystem** (in-memory, on local disk, or backed by a database, depending on deployment) that acts as external memory: large tool outputs get written to a file instead of sitting in the conversation, and the agent (or its sub-agents) can read files back later. This is what lets a task survive being paused, resumed, or handed to a different sub-agent without losing the work already done.



5. Context Engineering: Write, Select, Compress, Isolate

Everything above is really in service of one underlying problem: a model's context window is finite, and a long task naturally wants to fill it with junk (full tool outputs, repeated instructions, dead-end exploration). LangChain frames the discipline of managing this as **context engineering**, with four named strategies:

Strategy	What it means	Where it shows up in Deep Agents
Write	Decide what goes into the agent's context at startup.	The system prompt, persistent memory files (`AGENTS.md`), and "skills" — domain knowledge loaded only when relevant, not all at once.
Select	Decide what gets pulled in at runtime, on demand.	Per-call runtime context (user identity, API keys, session data) passed through a typed `context_schema` and automatically available to sub-agents too.
Compress	Shrink what's already in context before it overflows the window.	Automatic **offloading** — a tool result over ~20,000 tokens gets written to a file and replaced in-context with a reference — and automatic **summarization** once the conversation nears ~85% of the context limit, with the full original kept on disk and searchable.

Isolate	Keep heavy or messy sub-tasks from polluting the main thread at all.	Sub-agents: the main agent delegates via a `task` tool, and only a single final report returns — the intermediate mess never re-enters the main context.

> This is the actual engineering discipline behind "why doesn't this agent forget what it's doing halfway through a long task" — it is not that the model got smarter, it is that the system around the model was built to keep its working context clean.

6. Memory: Short-Term vs. Long-Term

Deep agents (and LangGraph underneath them) treat memory as two genuinely different things, and conflating them is a common source of confusion:

6.1 Short-term memory (within one run)

This is the state of one specific task while it's in progress — the conversation so far, the plan, intermediate results. LangGraph's **checkpointer** is what makes this durable: it snapshots the graph's state after each step, keyed to a **thread** (a specific run/conversation ID). If the process crashes, the machine restarts, or the run pauses for days waiting on a human, the checkpoint lets it resume from exactly where it left off — not from scratch.

6.2 Long-term memory (across runs)

This is knowledge that should outlive any single task — standing instructions, accumulated facts, prior decisions. Deep agents implement this through persistent files (an `AGENTS.md`-style memory file, loaded every time the agent starts) and a **composite backend** that routes specific file paths to durable storage instead of disappearing when the task ends.

> The short-term/long-term split maps almost exactly onto a distinction our own design already makes: a single enrollment case's in-progress state (short-term, resumable) versus the enrollment policy and audit history (long-term, persists across every case, forever). Section 10 makes this mapping explicit.

7. How a Deep Agent Actually Runs — A Walkthrough

Concretely, a single task moves through roughly this loop:

1. The agent receives a goal and writes an initial plan using the planning tool.
2. It works through the plan step by step, calling tools directly for light work, and delegating to a sub-agent for anything heavy or self-contained (e.g., "go read these five documents and summarize the relevant policy").
3. Large results get offloaded to the virtual filesystem automatically; only a reference stays in the active context.
4. If a step requires human sign-off, execution **pauses** (a LangGraph `interrupt()`) and the current state is checkpointed — the process can go idle, restart, or be deployed elsewhere, and the paused run is still there waiting.
5. Once the human responds (approve, reject, edit), the graph **resumes** from the exact checkpoint, not from the beginning.
6. The agent updates its plan, continues, and eventually reports a final result — with the full history of what happened auditable from the checkpoint log and the filesystem.

Notice that step 4–5 is, almost verbatim, the force-enrollment approval wait in our own design.

8. The LangChain Academy Deep Agents Course — Structure and Study Order

LangChain Academy runs a free course, *Introduction to Deep Agents*, structured as five modules. This is the recommended study order, with what each module is actually for:

Module	Focus	Why it matters for us
1	Building a deep agent — architecture, model selection, system prompts, tool integration, MCP, threading, checkpointing, human-in-the-loop.	This module alone covers the checkpointing and human-in-the-loop mechanics we need for the approval-token/instructor-approval step.
2	Execution environment — filesystem backends, sandboxes, shell/interpreter configuration.	Relevant mainly if we ever let the agent execute code directly; less central to a policy/approval workflow like ours.
3	Context management — summarization, offloading, skills, memory systems.	Directly informs how we should structure the enrollment policy corpus (as "skills," loaded on demand) rather than stuffing all policy into every prompt.
4	Delegation — subagent teams, dynamic subagent creation.	Relevant if the eligibility checks (prerequisites/seats/clashes) are ever split into isolated sub-agent calls rather than direct tool calls — see Section 10.
5	Putting it together — local deployment, a full sales-assistant project, async subagents.	The closest existing worked example to what our own end-to-end orchestrator needs to look like.

> A prerequisite worth naming: this course assumes basic familiarity with LangGraph (graphs, nodes, state). If any of that is unfamiliar,

> LangChain Academy's introductory LangGraph course is the natural thing to do first.

9. A Real-World Example: Open Deep Research (and Claude Code)

Open Deep Research is LangChain's own open-source deep-research agent, and it is the cleanest public example of the pattern end to end. At a high level: a supervisor agent takes a research question, writes a plan (topics to investigate), spawns parallel research sub-agents — each with its own isolated context — to go investigate one sub-topic each, and those sub-agents write their findings to a shared file system rather than returning raw search results. The supervisor then reads the condensed findings back, synthesizes them, and produces a final report. This is the isolate-and-compress strategy from Section 5, made concrete: none of the raw search noise from any single sub-agent ever touches the supervisor's own context.

Claude Code — the harness generating this very document — is the other standing public example, and arguably the more relevant one for us: it plans multi-step engineering tasks with a todo list, delegates self-contained investigations to sub-agents, reads and writes files as durable memory across a long session, and asks for human approval before consequential actions (exactly what tiered tool-approval settings are for). It is, in effect, a deep agent scoped to software engineering instead of research.

10. Technical Tie-In: Mapping Deep Agents onto the LUMS Process Orchestrator

This is the part that matters most for our actual build. The mapping below is deliberately literal — almost every deep-agent concept has a direct, already-designed counterpart in our own architecture:

Deep agent concept	Our project's equivalent	Note
Planning tool / write_todos	The orchestrator's internal plan for a case: extract intent → retrieve policy → check eligibility → route for approval → execute → notify.	We don't need the literal `write_todos` tool; the equivalent is that our orchestrator's plan is a known, fixed sequence per workflow type — closer to a pre-defined plan than a freely improvised one, which is actually safer for our approval-gated use case.
Checkpointing / threads	The `WORKFLOW_CASE` table — a case's state persists independently of any single chat session.	This is the single most direct mapping in this whole document. A LangGraph checkpointer keyed by case ID would *replace* a hand-rolled case-state table, and gets pause/resume for free instead of us re-implementing it.
Human-in-the-loop `interrupt()`	The instructor/HOD approval step before the enrollment write executes.	This is built into LangGraph natively — the graph pauses at the approval node, and resumes only once the instructor's decision arrives. We do not need to hand-build the "wait for days, then resume exactly here" logic; this is precisely what it's designed for.
Virtual filesystem / long-term memory	The policy RAG store, plus the audit log.	Policy documents as durable, versioned, on-demand-loaded context match the "skills" pattern almost exactly: don't load the whole policy corpus every turn, retrieve only the relevant section, cited.
Sub-agents (isolate strategy)	Candidate: the prerequisite/seat/clash eligibility check (UC-03) as an isolated call.	Important distinction: a deep-agent sub-agent is a *stateless worker with no independent approval authority* — it does one bounded task and returns a result. This is compatible with our decision to drop the earlier multi-specialist-agent design, because that design gave each specialist independent reasoning authority over a whole domain (Finance, Hostel, etc.). A deep-agent sub-agent is narrower and safer: it cannot approve anything, cannot write anything, and reports back to the one orchestrator that still holds all authority. Adopting this pattern would not be a reversal of our single-orchestrator decision.
Context compression (offload/summarize)	Not yet designed, but relevant once cases run long or a student has many open cases.	Worth planning for during Phase 2 rather than the current MVP.

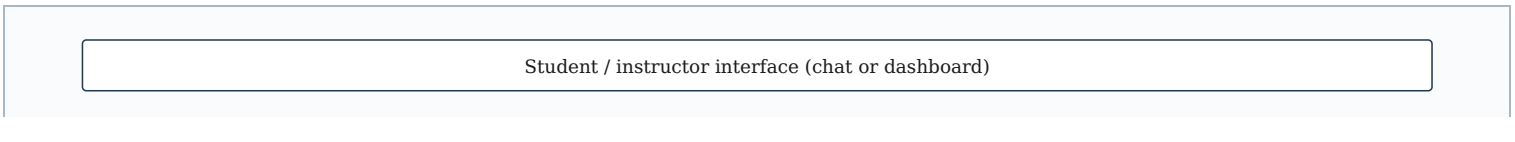
> The practical upshot: our own architecture, designed independently across several proposal drafts, already converges on the shape of a deep agent. That is worth saying to Sir directly — it is evidence the design is grounded in a real, published engineering pattern, not an ad hoc invention.

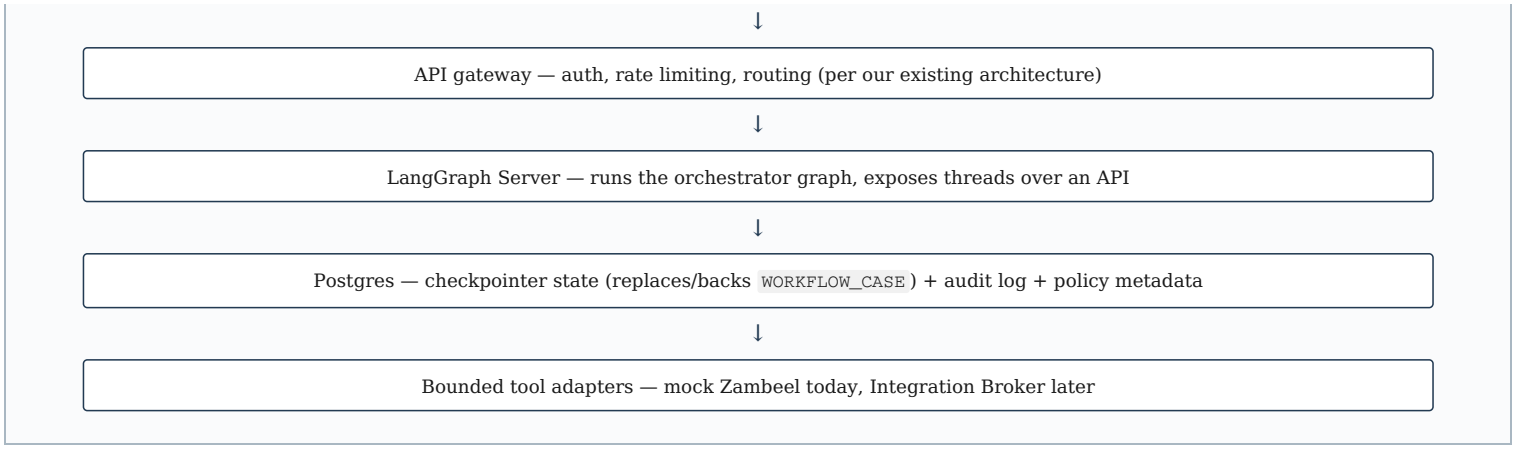
11. Deployment: How This Would Actually Run in Production

Deep agents built on LangGraph are typically deployed one of two ways:

- **LangGraph Platform** (managed) — LangChain's own hosted deployment service, with built-in persistence, streaming, and human-in-the-loop support over an API, plus native LangSmith tracing.
- **Self-hosted LangGraph Server** — the same runtime, run on our own infrastructure, backed by our own Postgres database for checkpoints.

For our project specifically, self-hosting is the realistic option, since a self-hosted deployment sits comfortably inside LUMS's own network boundary — which matters given the earlier finding that Zambeel's integration endpoint is only reachable from inside that network in the first place. A plausible production shape:





The approval step becomes concrete under this model: the graph reaches the approval node and calls `interrupt()`, which pauses the thread and persists its state to Postgres. The instructor's approval (arriving through whatever channel — email link, dashboard, or a notification agent) resumes that exact thread by ID. No polling loop or custom "check if approved yet" logic is needed — LangGraph's resume mechanism does this directly.

For observability, LangSmith (or a self-hosted equivalent trace store) gives per-step tracing of every tool call, every plan revision, and every interrupt — which is a natural fit for the "complete audit record for every consequential action" requirement already written into our BRD's acceptance criteria and NFR-05.

Containerization follows the same pattern as any other backend service: the LangGraph server and Postgres each run as containers, which is exactly the NFR-08 item ("containerization and observability") already flagged as a Phase 2 concern in our requirements document.

12. Honest Trade-Offs — Is the Full Framework Worth Adopting for Our MVP?

It would be dishonest to present this as a free upgrade with no cost, so a few things worth weighing before committing to the `deepagents` package specifically, as opposed to just the underlying *pattern*:

- **Our Phase 1 workflow is linear, not exploratory.** Force enrollment has one fixed sequence of steps, unlike an open-ended research task. A lot of the deep-agent machinery (dynamic sub-agent spawning, freeform planning) is built for tasks where the path isn't known in advance. We may only need a thin slice of the full framework — principally the checkpointer and the human-in-the-loop interrupt — not the whole harness.
- **LangGraph alone (without the `deepagents` package) may be the better fit.** We could hand-build our fixed workflow directly as a LangGraph graph and get checkpointing and interrupts natively, without taking on the sub-agent/filesystem/skills machinery we don't yet need. That keeps the system smaller and easier to fully audit — which matters given our own "zero unauthorized writes" and full-auditability requirements.
- **Adopting a framework is a real dependency, not a neutral choice.** It shapes our deployment (needs LangGraph Server or Platform), our team's learning curve, and our debugging surface. This is worth being explicit about if asked why we did or didn't adopt the package directly, rather than presenting it as an obviously correct choice.
- **The pattern generalizes better than the package does.** Sir's instruction to build a "generic, configurable workflow engine" (Sept 8 notes) is arguably better served by understanding *why* deep agents are structured this way and building our own lean version of it, than by importing a general-purpose harness built for open-ended research tasks.

> Recommendation: study the pattern thoroughly (this document plus the Academy course), but treat "do we import the `deepagents` package" and "do we build directly on LangGraph with checkpointing + interrupts" as a real, separate decision to bring to Sir — not settled by default.

13. Glossary

Agent
An LLM given tools and permission to decide, in a loop, which tool to call next until it judges the task done.

Agentic loop / tool-calling loop
The basic cycle: model reasons → calls a tool → reads the result → reasons again, repeated until finished.

Checkpoint / checkpointer
A saved snapshot of a graph's exact state at a point in time, keyed to a thread, letting execution pause and resume later.

Thread
A specific, ongoing run of a graph (e.g., one specific enrollment case), identified by an ID so its state can be resumed.

Interrupt
A LangGraph mechanism that pauses execution at a specific point — typically to wait for a human decision — and resumes from that exact point once a response arrives.

Sub-agent
A short-lived child agent given one narrow task and a clean context window, which reports back a single condensed result.

Context window

The finite amount of text (measured in tokens) a model can consider at once in a given call.

Context engineering
The discipline of deciding what goes into, stays in, or is removed from an agent's context window over a long task.

MCP (Model Context Protocol)
A standard way to expose tools and data sources to an agent so different agents/frameworks can use the same tool definitions.

RAG (Retrieval-Augmented Generation)
Retrieving relevant documents (e.g., a policy section) and grounding the model's answer in them, with a citation, instead of relying on memorized knowledge.

14. Sources

- LangChain — Deep Agents overview: https://docs.langchain.com/oss/python/deepagents/overview
- LangChain — Context engineering in Deep Agents: https://docs.langchain.com/oss/python/deepagents/context-engineering
- LangChain — Deep Agents (product/blog page): https://www.langchain.com/deep-agents
- LangChain blog — Deep Agents (origin, four pillars, Claude Code/Manus/Deep Research inspiration): https://www.langchain.com/blog/deep-agents
- LangChain Academy — Introduction to Deep Agents course: https://academy.langchain.com/courses/foundation-introduction-to-deepagents
- GitHub — langchain-ai/deepagents (README, installation, usage): https://github.com/langchain-ai/deepagents
- LangChain blog — Open Deep Research: https://blog.langchain.com/open-deep-research/
- LangChain — LangGraph Persistence docs: https://docs.langchain.com/oss/python/langgraph/persistence
- LangGraph checkpoint-postgres README: https://github.com/langchain-ai/langgraph/blob/main/libs/checkpoint-postgres/README.md